

Parallel Computing Efficiency

A project report
submitted by

Jacob Martin
(2022MTH554**)

under the supervision of
Dr. Chuan Li



Department of Mathematics
West Chester University
December 2022

Contents

0.1	Introduction	2
0.2	Methods	3
0.3	Experiments	5
0.4	Structure	10
0.5	Conclusions	13

0.1 Introduction

The purpose of this report is meant to analyze the accuracy, performance, and convergence properties of various numerical techniques for boundary value problems type second order linear differential equations. We will also analyze the efficiency and computational cost of techniques developed in a sequential manner vs the same techniques developed in a parallel manner. For parallel algorithms, the MPI library was used in Fortran90. Examined equations in this report are in the following format:

$$y'' = p(x)y' + q(x)y + r(x)$$

$$\text{where } a \leq x \leq b, \text{ with } y(a) = \alpha \text{ and } y(b) = \beta$$

Generally speaking, parallel computing is extremely useful in today's scientific community. Parallel computing can be used in various industries such as predictive analytics, weather pattern mapping, disease simulations, and much more. The main advantage that parallel computing gives to the user is time. With proper implementation, substantial time can be saved compared to its serial counterpart. In a professional setting, time is money, and parallel computing can be well worth the cost for the amount of time it saves.

Implementing numerical techniques to compute in parallel does come with a cost, the main being the complexity that it adds to a given project. It is paramount to write in a way that can make sense to yourself, and also any other potential developers. Debugging can be complex and the feedback loop long. Development of parallel computing algorithms requires a strong fundamental understanding of standard programming practices and structures. To highlight this point, in this report we will also examine the structural complexity of the parallel implementation vs the structural complexity of the sequential implementation.

0.2 Methods

Jacobi Iterative Method

The first method this report examines is the Jacobi iterative method. The Jacobi is an iterative technique used to solve $n \times n$ linear systems in the form $A\mathbf{x} = \mathbf{b}$. The method starts with an initial approximation of $\mathbf{x}^{(0)}$ to the solution $\mathbf{x}$ and generates a sequence of vectors $(\mathbf{x}^k)_{k=0}^{\infty}$ that converges to $\mathbf{x}$. Recall that the Jacobi iterative method is obtained by solving the i th equation in $A\mathbf{x} = \mathbf{b}$ for x_i to obtain

$$x_i = \sum_{j=1, j \neq i}^n \left(-\frac{a_{i,j}x_j}{a_{i,i}} \right) + b_i a_{i,i} \text{ for } i = 1, 2, \dots, n.$$

For each $k \geq 1$, generate the components $x_i^{(k)}$ of $\mathbf{x}^{(k)}$ from the components of $\mathbf{x}^{(k-1)}$:

$$x_i^{(k)} = \frac{1}{a_{i,i}} \left[\sum_{j=1, j \neq i}^n \left(-a_{i,j}x_j^{(k-1)} + b_i \right) \right], \text{ for } i = 1, 2, \dots, n. \quad [1]$$

Crout Factorization Method

The second method this report examines is the Crout Factorization method. The method is not iterative, but rather directly solves for $\mathbf{x}$ by capitalizing on the tridiagonal property of matrix $\mathbf{A}$. The method involves breaking down $\mathbf{A}$ into triangular matrices $\mathbf{L}$ and $\mathbf{U}$. Where $\mathbf{L}$ and $\mathbf{U}$ also have tridiagonal form. The algorithm can be applied to any matrix whenever $L_{i,i} \neq 0$ for each $i = 1, 2, \dots, n$. $\mathbf{L}$ also needs one of the two following conditions to be true for the algorithm to run successfully. (1) The system must be positive definite or (2) the system must be strictly diagonally dominant. With those properties satisfied, $\mathbf{A}$ can be found via the following non-iterative method,

$$a_{1,1} = L_{1,1}$$

$$\begin{aligned} a_{i,i-1} &= L_{i,i-1}, \text{ for each } i = 2, 3, \dots, n; \\ a_{i,i} &= L_{i,i-1}U_{i-1,i} + L_{i,i}, \text{ for each } i = 2, 3, \dots, n; \\ a_{i,i+1} &= L_{i,i}U_{i,i+1}, \text{ for each } i = 2, 3, \dots, n-1. \end{aligned}$$

Equation examined in this report

The equation that is used in measuring the efficiency of each aforementioned method is the following problem taken from section 11.3 of the textbook, *Numerical Analysis, 9th Edition*,

$$y'' = -\frac{2}{x}y' + \frac{2}{x^2}y + \frac{\sin(\ln(x))}{x^2}$$

$$\text{for } 1 \leq x \leq 2, \text{ with } y(1) = 1 \text{ and } y(2) = 2$$

The exact answer to the differential equation is given as,

$$y = c_1x + \frac{c_2}{x^2} - \frac{3}{10}\sin(\ln(x)) - \frac{1}{10}\cos(\ln(x)),$$

where

$$c_1 = \frac{11}{10} - c_2,$$

and

$$c_2 = \frac{1}{70} (8 - 12\sin(\ln(2)) - 4\cos(\ln(2))).$$

Lastly, recall the format mentioned in the introduction and translate it to be relevant to the problem at hand,

$$p(x) = \frac{-2}{x}y', \quad q(x) = \frac{2}{x^2}y, \quad r(x) = \frac{\sin(\ln(x))}{x^2} \text{ with } a = 1.0, \quad b = 2.0 \text{ and } y(a) = 1.0, \quad y(b) = 2.0.$$

0.3 Experiments

For the following experiments, n will determine the number of mesh-points to be approximated. Let M be the max number of potential iterations, this is set to 2,000,000 for every experiment. Allow w_i to be the approximated value at x_i , and y_i to be the exact value at x_i . Finally, let C denote the number of iterations it took for the jacobi method to converge and let T denote the tolerance. To maximize computational efficiency only mesh-points 1, 1.25, 1.5, 1.75, and 2.0 were printed out when running all experiments. Lastly, the sequential and jacobian output for accuracy are identical. Therefore, both of these methods will share one output table. When applicable, *Jacobi_S* will denote the sequential version and *Jacobi_P* will denote the parallel version. Let's start with examining tables 1, 2, and 3.

Table 1: Crout, $n = 19$, $T = 1 \times 10^{-7}$

x_i	w_i	y_i	err
1.0	1.0	1.0	0.0
1.25	1.23499191	1.23500669	1.47819519E-05
1.5	1.48114502	1.48115957	1.45435333E-05
1.75	1.73679745	1.73680592	8.46385956E-06
2.0	2.0	2.0	0.0

Table 2: Jacobi, $n = 19$, $T = 1 \times 10^{-7}$

x_i	w_i	y_i	err
1.0	1.0	1.0	0.0
1.25	1.23496187	1.23500669	4.48226929E-05
1.5	1.48110855	1.48115957	5.10215759E-05
1.75	1.73677576	1.73680592	3.01599503E-05
2.0	2.0	2.0	0.0

Table 3: Defining Metrics

Method	Runtime	C	L^2error
Crout	0m0.003s	N/A	1.15043867E-05
Jacobi_S	0m0.003s	802	3.82816106E-05
Jacobi_P	0m0.031s	802	3.82816106E-05

The results of the first experiment yield all methods to be accurate and efficient. This is to be expected as all required mathematical properties hold true for the matrix $\mathbf{A}$ to converge. While all three methods converge in less than a second, it is important to note that with an n size so small, the added overhead that comes along with MPI point to point communication between processes makes the parallel Jacobi perform slower than it's sequential counterpart. Let's continue analyzing data by examining tables 4, 5, and 6.

Table 4: Crout, $n = 999$, $T = 1 \times 10^{-7}$

x_i	w_i	y_i	err
1.0	1.0	1.0	0.0
1.25	1.23546898	1.23500669	-4.62293625E-04
1.5	1.48149693	1.48115957	-3.37362289E-04
1.75	1.73679876	1.73680592	7.15255737E-06
2.0	2.0	2.0	0.0

Table 5: Jacobi, $n = 999$, $T = 1 \times 10^{-7}$

x_i	w_i	y_i	err
1.0	1.0	1.0	0.0
1.25	1.15272057	1.23500669	8.22861195E-02
1.5	1.38239801	1.48115957	9.87615585E-02
1.75	1.67492330	1.73680592	6.18826151E-02
2.0	2.0	2.0	0.0

Table 6: Defining Metrics

Method	Runtime	C	L^2error
Crout	0.005s	N/A	3.65478569E-04
Jacobi_S	11.647s	562,533	1.34096649E-02
Jacobi_P	4.749s	562,533	1.34096649E-02

In this particular experiment, the crout factorization method is the most efficient while also the most accurate. This is to be expected as it is not an iterative method, rather a direct method, which requires much less computations. Furthermore, when comparing the sequential Jacobi against the parallel Jacobi, $\approx 250\%$ increase in processing time is achieved! This is due to the nature of parallel computing which is further described in section four. Next, let's examine tables 7, 8, and 9.

Table 7: Crout, $n = 99999$, $T = 1 \times 10^{-7}$

x_i	w_i	y_i	err
1.0	1.0	1.0	0.0
1.25	1.00438285	1.23500669	0.272542119
1.5	1.14382648	1.48115957	0.337333083
1.75	1.46426380	1.73680592	0.230623841
2.0	2.0	2.0	0.0

Table 8: Jacobi, $n = 99999$, $T = 1 \times 10^{-7}$

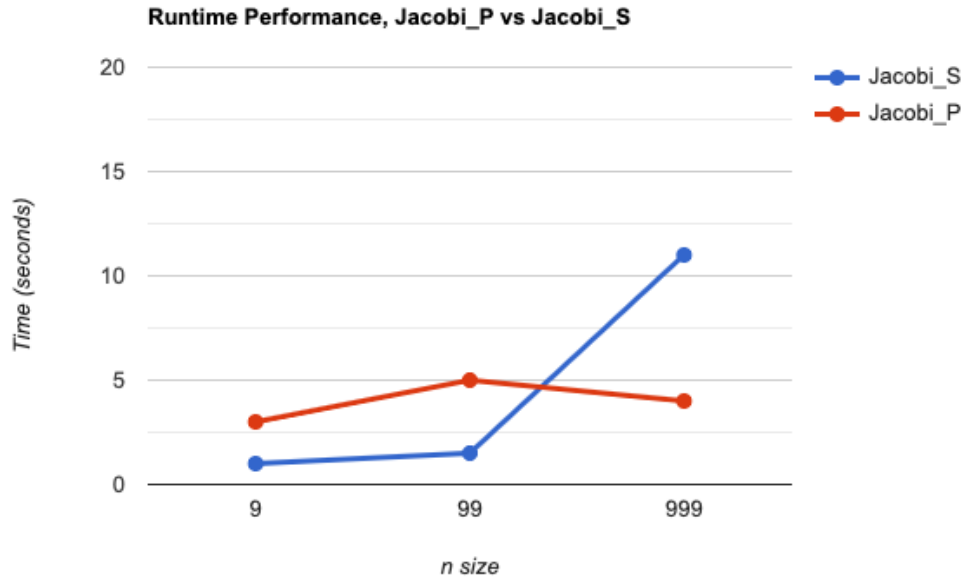
x_i	w_i	y_i	err
1.0	1.0	1.0	0.0
1.25	-6.92302456E-06	1.23500669	1.23501360
1.5	-8.53318306E-06	1.48115957	1.48116815
1.75	-8.45770410E-06	1.73680592	1.73681438
2.0	2.0	2.0	0.0

Table 9: Defining Metrics

Method	Runtime	C	L^2error
Crout	0.025s	N/A	0.246343672
Jacobi_S	17m40.71s	973,813	1.49690974
Jacobi_P	7m54.585s	973,813	1.49690974

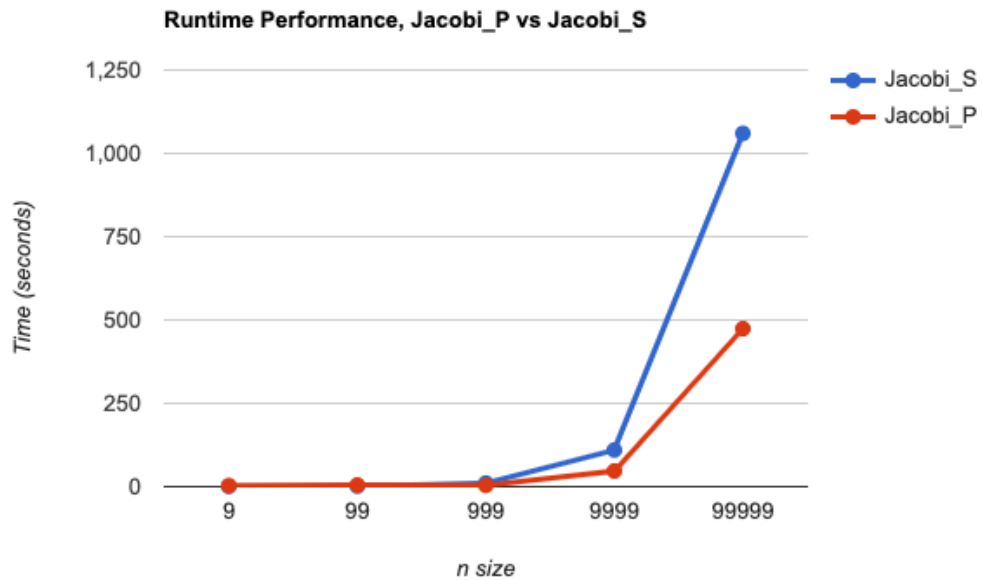
These results hold interesting properties. The accuracy of all methods have dropped off considerably. While the method still converges, the approximated solution is vastly incorrect in numerical standards. This is mainly because with each iteration there is an error that starts to accumulate. This error get carried over and increases with each iteration. Recall that $a = 1.0$ and $b = 2.0$. This is a relatively small length for a high value of n . Therefore the error accumulates dramatically and our method fails to be accurate. It is still impressive to note the time savings the parallel Jacobi method brings into the fold; it still maintains $\approx 223\%$ performance increase.

Furthermore, the graph below identifies exactly when parallel Jacobi becomes more efficient to run vs the sequential Jacobi.



Clearly we can see around 500 iterations is when the parallel Jacobi becomes more

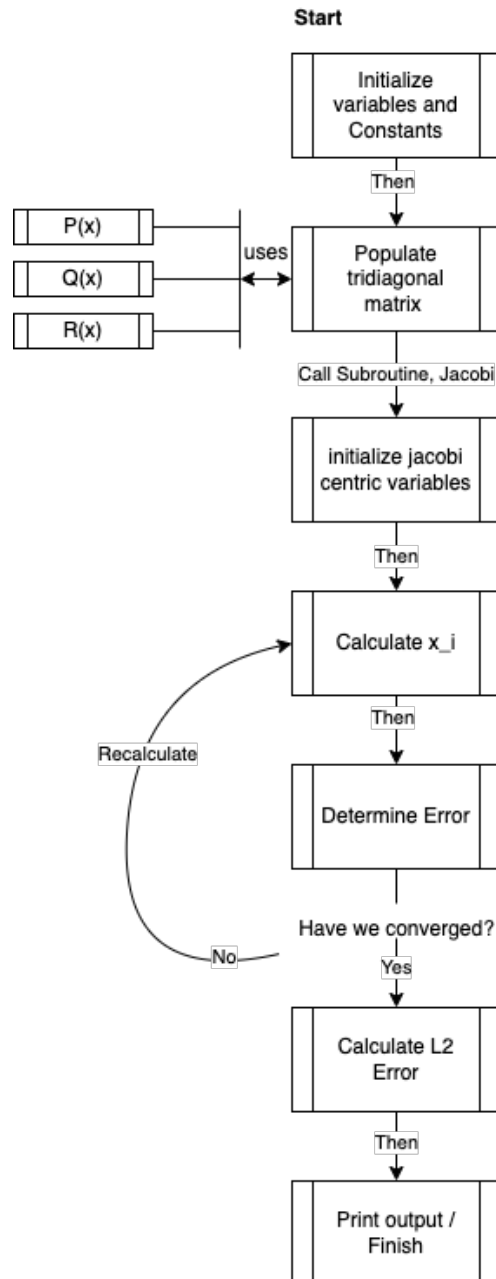
efficient to run. Lastly, let's determine if the overall run-time of the Jacobi behaves linearly or exponentially. Note the graph below.



Clearly this behaves exponentially. This is because as n increases, so does our matrix $\mathbf{A}$. As $\mathbf{A}$ gets large, the computational requirements increase exponentially.

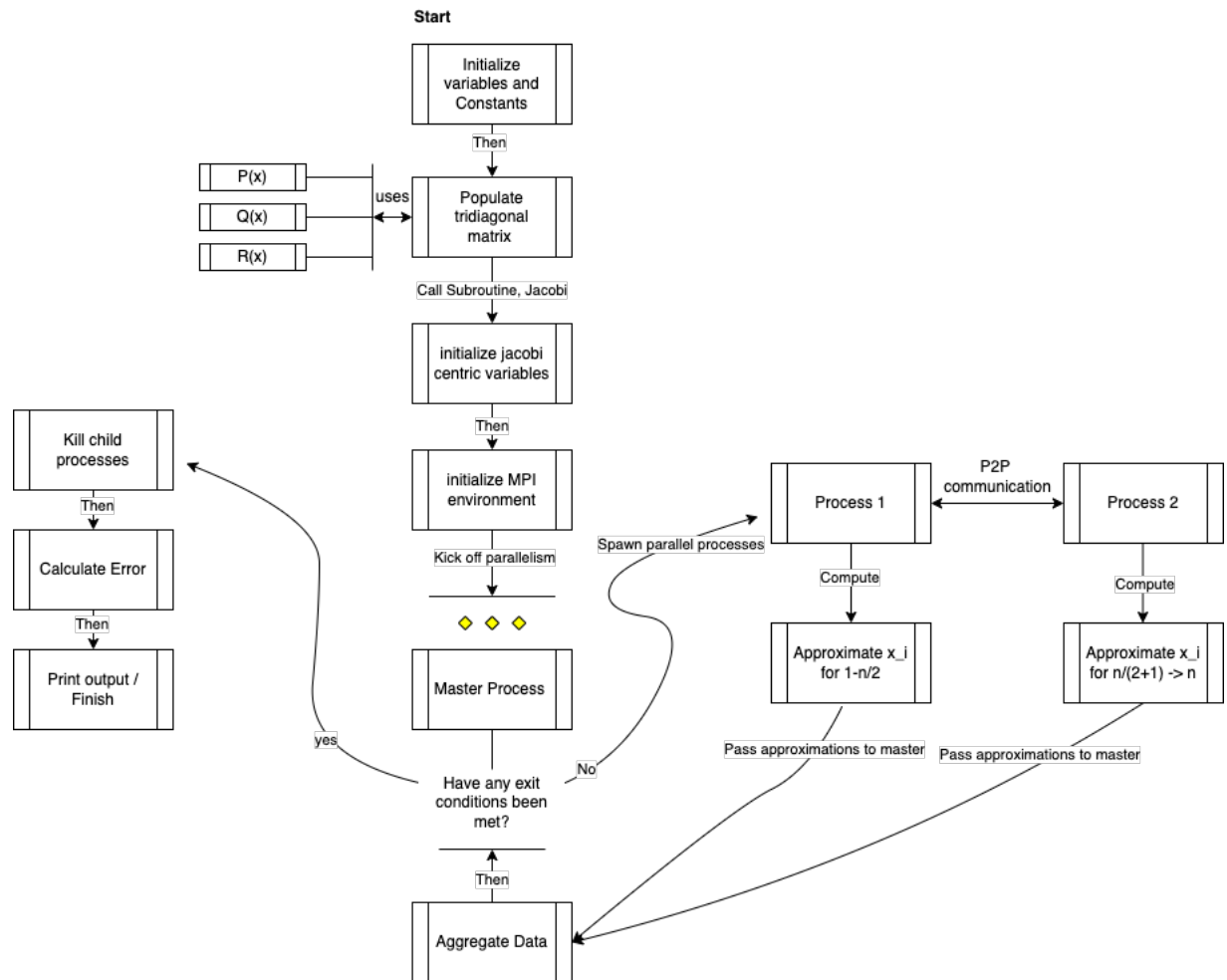
0.4 Structure

As previously mentioned, the structure of the code where the Jacobi induces parallelism is substantially more complicated to develop. To understand why, let's start by analyzing the structure of the sequential implementation of the Jacobi:



As we can see, the structure is pretty straightforward. Typical variable instantiating occurs, then we build the tridiagonal matrix using $p(x)$, $q(x)$, and $r(x)$. Once we are within the subroutine of the Jacobi method, it is important to note that there is only one outer control structure dictating the way the method approximates. Aside to that, the calculations are standard to the definition of the method. Lastly, we calculate the L^2 error and print the output.

Now that we have analyzed the sequential implementation, let's look at the structure of the parallel computing implementation for the Jacobi:



As we can see, the structure at start of the parallel algorithm is identical to the start of the sequential algorithm. However, there is added complexity in initializing the MPI environment, along with deciding how the master process will

control the other processes. While the sequential implementation has only one outer control structure dictating how the fundamental algorithm operates, this implementation has three main outer control structures, one in the master process and one in each of the child processes. All of these outer most control structures must be appropriately handled to ensure an efficient algorithm. In addition, there is added complexity in the implementation of point to point communication within the child processes. These processes need critical values to continue approximating, therefore the processes must stay in constant communication with each other syncing metadata. Furthermore, the master process also needs to aggregate the approximated data and make it's own calculations and decisions. The decisions that are made from this data are vital to controlling the fundamental algorithm as a whole. All of these programming challenges add to the complexity of the parallel computing paradigm, especially when compared to the sequential implementation.

Something worth noting with this implementation is that there is still a lot of room for improvement. While there are beneficial time savings and added efficiency with the implementation, the code itself is not as parallelized as it could be. For example, consider the construction of the tridiagonal matrix. Given a large enough matrix, this step could be ran in parallel to improve the efficiency of matrix creation. Lastly, the manner in which child processes use blocking point to point communication with the master process could be improved upon to send more data in one message. This would reduce the number of blocking calls to the outer most control structure which in theory could reduce the overall run time of the algorithm.

0.5 Conclusions

There are a few main takeaways from this project. The first and most interesting being that numerical methods can be pretty sensitive on how they behave depending on the parameters that are given to the method. In section three, a small boundary condition with high n size was demonstrated to throw off the accuracy of the method, while still allowing it to converge. With more time permitted, it would be interesting to see the behaviour of the method over a larger interval. It seems there is a balance to the length of the interval and the size of n which would cause the methods to be computationally efficient and accurate. The second takeaway is that developing parallel computing algorithms is not simple, but provide immense time savings when done correctly. This is important in professional settings as individuals who possess the skills to do this in a scientific manner are few and far between. These skills are highly sought after and valued amongst the professional software industry. Altogether, even though it is more complicated to implement parallel computing, it is a skill that is well worth it to learn. Finally, coding with Fortran90 proved to be useful. It really requires the programmer to fundamentally understand what they are doing as documentation is pretty sparse for the language. This class and project are beneficial because it improves raw coding skills to where one can be semi-fluent in reading and writing Fortran90.

Bibliography

- [1] Burdon, Richard L. & Faires, J. Douglas. *Numerical Analysis, Ninth Edition*
Richard Stratton, (2010), pg. 450-451.